

实验 1 卷积神经网络 CNN 完成图像分类任务实验

一、 实验目的

理解并掌握基于卷积神经网络的图像分类任务。

二、 实验内容

利用 python 语言编写卷积神经网络代码。实验使用 Kaggle 竞赛平台上的 Flower Recognition 数据集

(<https://www.kaggle.com/axmamaev/flowers-recognition>), 包含 5 种花卉种类, 总计 4326 张标记过的图片。通过搭建多层卷积神经网络进行监督学习完成分类任务。

三、 实验环境

硬件: x86_64 Centos 3.10.0 服务器、GPU 服务器、PC 上位机;
python3.5、Jupyter Notebook、Kaggle 数据集。

四、 实验原理

卷积神经网络 (Convolutional Neural Network, 简称 CNN), 是一种前馈神经网络, 人工神经元可以响应周围单元, 可以进行大型图像处理。卷积神经网络包括卷积层和池化层。

CNN 网络一共有 5 个层级结构:

1. 输入层
2. 卷积层
3. 激活层
4. 池化层
5. 全连接 FC 层

输入层:

与传统神经网络/机器学习一样, 模型需要输入的进行预处理操作, 常见的 3 种预处理方式有:

1. 去均值
2. 归一化
3. PCA/SVD 降维等

卷积层:

局部感知：人的大脑识别图片的过程中，并不是一下子整张图同时识别，而是对于图片中的每一个特征首先局部感知，然后更高层次对局部进行综合操作，从而得到全局信息。

权值共享：卷积层使用“卷积核”进行局部感知。对于每一层来讲，所有神经元对应的权值是相等的。

激励层：

所谓激励，实际上是对卷积层的输出结果做一次非线性映射。

如果不用激励函数（其实就相当于激励函数是 $f(x)=x$ ），这种情况下，每一层的输出都是上一层输入的线性函数。容易得出，无论有多少神经网络层，输出都是输入的线性组合，与没有隐藏层的效果是一样的，这就是最原始的感知机了。

常用的激励函数有：

1. Sigmoid 函数
2. Tanh 函数
3. ReLU
4. Leaky ReLU

池化层：

池化（Pooling）：也称为欠采样或下采样。主要用于特征降维，压缩数据和参数的数量，减小过拟合，同时提高模型的容错性。主要有：

1. Max Pooling：最大池化
2. Average Pooling：平均池化

输出层：

经过前面若干次卷积+激励+池化后，终于来到了输出层，模型会将学到的一个高质量的特征图片全连接层。其实在全连接层之前，如果神经元数目过大，学习能力强，有可能出现过拟合。因此，可以引入 dropout 操作，来随机删除神经网络中的部分神经元，来解决此问题。还可以进行局部归一化（LRN）、数据增强等操作，来增加鲁棒性，这里不做介绍。

当来到了全连接层之后，可以理解为一个简单的多分类神经网络（如：BP 神经网络），通过 softmax 函数得到最终的输出。整个模型训练完毕。

五、 实验分析及步骤

本实验使用了 Kaggle 竞赛平台上的 Flower Recognition 数据集，包含 5 种花卉种类，总计 4326 张标记过的图片。利用 Python 搭建 CNN 网络，对这五种类别图像进行分类。在实验之前首先将 flowers 文件夹下的五个文件夹重命名为 0, 1, 2, 3, 4 表示五中花卉。神经网络框架使用 PyTorch，PyTorch 是一个开源的 Python 机器学习库，基于 Torch。它是一个基于 Python 的可续计算包，提供两个高级功能：具有强大的 GPU 加速的张量计算；包含自动求导系统的深度神经网络。

1、准备数据集

在进行网络训练之前需要对数据集进行预处理，包括将数据集划分为训练集和验证集，数据归一化等等。数据预处理的目的是将数据从原始的图片、标注转换为机器学习所需的规范化之后的数据。

数据预处理使用 torchvision.transforms 模块。torchvision.transforms 是 pytorch 中的图像预处理包，包含了很多对图像数据进行变化的模块。我们在实验中主要用到的方法有 torchvision.transforms.Compose(transforms) 方法，将多种变换组合在一起；torchvision.transforms.Resize(size) 将图像 resize 到指定的 size；torchvision.transforms.ToTensor() 将 PIL 图片转换为 tensor；torchvision.transforms.normalize() 数据归一化。

```
normalize = transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])
transformer_Image = transforms.Compose([
    transforms.Resize((28, 28)),    #resize 图像为 28*28，太大的话需
要的训练时间太长
    transforms.ToTensor(),
    normalize
])
```

2、生成数据

加载自己的数据时需要继承抽象类 torch.utils.data.Dataset，并重写其中的两个方法：__len__ 返回整个数据集的大小，__getitem__ 用来获取一些索引的书籍，使 dataset[i] 返回数据集中第 i 个样本。

```
class FlowerDataset(Dataset):
```

```

def __init__(self, filenames, labels, transform):
    self.filenames = filenames
    self.labels = labels
    self.transform = transform

def __len__(self):
    return len(self.filenames)

def __getitem__(self, idx):
    image = Image.open(self.filenames[idx]).convert('RGB')
    image = self.transform(image)
    return image, self.labels[idx]

```

`split_train_val_data(data_dir, ratio)` 函数将数据集划分为训练集和验证集，其中 `data_dir` 为数据存放目录，`ratio` 为训练集和验证集的比例。在划分数据集时主要使用到 `torchvision.datasets.ImageFolder` 方法，用来加载如下组织的数据集。

```

flowers/0/xxx.jpg
flowers/0/xyy.jpg
...
Flowers/4/xxz.jpg

```

`DataLoader` 函数帮我们实现是否打乱数据，数据划分为 `batch` 等操作。定义如下：

```

def __init__(self, dataset, batch_size=1, shuffle=False,
sampler=None, batch_sampler=None, num_workers=0, collate_fn=None,
pin_memory=False, drop_last=False, timeout=0, worker_init_fn=None,
multiprocessing_context=None)

```

其中 `dataset`：加载的数据集；

`Batch_size`：一个 `batch` 包含的数据数；

`Shuffle`：是否将数据打乱；

其它参数本实验中使用默认值（如果选择在 EduCoder 平台做实验，请不要修改代码中的文件路径）。

#划分数数据集为训练集和验证集，数据以 zip 压缩包的格式存放在 EduCoder 平台，所以需要借助 zipfile 模块读取，如果选择在 EduCoder 平台做实验，请不要修改代码中的文件路径

```
def split_train_val_data(data_dir, ratio):
    #the sum of ratio must equal to 1
    file=zipfile.ZipFile('/data/shixunfiles/a95dd23359bce6dcae30d82349f94a14_1586277976202.zip', 'r')
    file.extractall('/data/Experiment5/')
    dataset = ImageFolder('/data/Experiment5/'+file.namelist()[0]) #
data_dir 精确到分类目录的上一级
    character = [[] for i in range(len(dataset.classes))]
    # print(dataset.class_to_idx)
    for x, y in dataset.samples: # 将数据按类标存放
        character[y].append(x)
    # print(dataset.samples)
    train_images, val_images = [], []
    train_labels, val_labels = [], []
    for i, data in enumerate(character): # data 为一类图片
        num_sample_train = int(len(data) * ratio[0])
        # print(num_sample_train)
        for x in data[:num_sample_train]:
            train_images.append(x)
            train_labels.append(i)
        for x in data[num_sample_train:]:
            val_images.append(x)
            val_labels.append(i)
    # print(len(train_inputs))
```

```

train_dataloader = DataLoader(FlowerDataset(train_images,
train_labels, transformer_Image),
                             batch_size=28, shuffle=True)

val_dataloader = DataLoader(FlowerDataset(val_images, val_labels,
transformer_Image),
                             batch_size=len(val_labels),
shuffle=False)

return train_dataloader, val_dataloader

```

3、搭建模型

实验中我们使用的卷积神经网络包含两个卷积层，三个全连接层，最后一个 softmax 做分类。在模型搭建中我们使用 torch.nn.Conv2d 和 torch.nn.Linear 接口分别搭建卷积层和全连接层。

PyTorch 中 Conv2d 定义如下：

```

def __init__(self, in_channels, out_channels, kernel_size, stride=1,
              padding=0, dilation=1, groups=1,
              bias=True, padding_mode='zeros'):

```

In_channels: 输入通道数，对于 RGB 图像，输入层的输入通道数为 3；

Out_channels: 输出通道数，和下一层卷积层的输入通道数相等；

Kernel_size: 卷积核大小；

Stride: 卷积步长；

Padding: 默认为 0；

激活函数选用 ReLU()，池化层选用最大池化，卷积核为 2*2；

由此可得第一个卷积层定义如下：

```

self.conv1 = nn.Sequential(
    nn.Conv2d(
        in_channels=3,
        out_channels=24,
        kernel_size=3,
        stride=1,

```

```

        padding=1,
    ),
    #激活函数为 Rule()
    nn.ReLU(),
    #最大池化, Kernel 为 2*2
    nn.MaxPool2d(kernel_size=2),
)

```

补充剩余的卷积层和全连接层代码:

```

class CNN(nn.Module):
    def __init__(self, in_channels, n_classes):
        super(CNN, self).__init__()
        #第一个卷积层
        self.conv1 = nn.Sequential(
            nn.Conv2d(
                in_channels=in_channels,
                out_channels=24,
                kernel_size=3,
                stride=1,
                padding=1,
            ),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2),
        )
        /**
        *补充第二个卷积层代码
        *in_channels=24, out_channels=12, kernel_size=3, stride=1,
        *padding=1
        *激活函数为 Relu(), 池化层为最大池化, 卷积核为 2*2
        */

```

```

#第一个全连接层
self.fc1 = nn.Sequential(
    nn.Linear(in_features=12 * 7 * 7, out_features=196),
    nn.ReLU(),
)

/**
*补充第二个全连接层代码
*in_features=196, out_features=84
*激活函数为 Relu()
*/

/**
*补充第三个全连接层代码
*in_features=84, out_features=n_classes
*/

def forward(self, x):
    x = self.conv1(x)
    x = self.conv2(x)
    #print("x.shape", x.shape)
    x = x.view(-1, 12*7*7)
    x = self.fc1(x)
    x = self.fc2(x)
    x = self.fc3(x)
    output = F.softmax(x, 1)
    return output

```

4、模型评估

Loss

Loss 函数计算预测值和实际值之间的差距，采用了交叉熵函数，用于反向

传播进行参数优化。

```
loss_func = nn.CrossEntropyLoss()
```

优化器使用 adam 进行了优化。Adam 中动量直接并入了梯度一阶矩（指数加权）的估计。其次，相比于缺少修正因子导致二阶矩估计可能在训练初期具有很高偏置的 RMSProp，Adam 包括偏置修正，修正从原点初始化的一阶矩（动量项）和（非中心的）二阶矩估计。

```
optimizer = torch.optim.Adam(cnn.parameters(), lr=learning_rate)
```

计算模型在验证集上的准确率，训练过程中，通过训练到一定阶段的模型预测验证集上分类结果，并计算准确率。

5、训练模型

首先从本地加载数据集，分成 batch。然后定义各部分训练，EPOCH 为最大训练次数，EPOCH=10 也就是训练了 10 次，learning_rate 为学习率。

最后在 main 函数中进行训练，并输出训练过程中训练集上的损失值和模型在验证集上的准确率。

```
# 变量声明
N_CLASSES = 5    #花卉种类数
BATCH_SIZE = 20  # 每个 batch 要放多少张图片
EPOCH = 10
learning_rate = 0.001
data_dir = "data/flowers"
ratio = [0.8, 0.2] #训练集：验证集

if __name__ == "__main__":

    train_loader, val_loader = split_train_val_data(data_dir, ratio,
batch_size=BATCH_SIZE)

    cnn = CNN(in_channels=3, n_classes=N_CLASSES)
    optimizer = torch.optim.Adam(cnn.parameters(), lr=learning_rate)
```

```

loss_func = nn.CrossEntropyLoss()

for epoch in range(EPOCH):
    train_accuracy = []
    for step, (x, y) in enumerate(train_loader):
        b_x = Variable(x)
        b_y = Variable(y)
        output = cnn(b_x)
        loss = loss_func(output, b_y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        if step % 10 == 0:
            #计算模型在验证集上的准确率
            val_accuracy = []
            for (data, target) in val_loader:
                val_output = cnn(data)
                pred = torch.max(val_output, 1)[1]
                val_acc = sum(pred == target)
                val_num = len(target)
                val_accuracy.append([val_acc, val_num])
            # val_r 为一个二元组，分别记录校验集中分类正确的数量
            和该集合中总的样本数
            val_r = (sum([tup[0] for tup in val_accuracy]),
sum([tup[1] for tup in val_accuracy]))
            print('Epoch: ', epoch, '|train loss: %.4f' % loss, '
验证集上准确率: ', (val_r[0].float()/val_r[1]).numpy())

torch.save(cnn, 'model/model.pkl') #保存模型

```

6、训练过程

随着训练步数的增加，loss 在减小，validate accuracy 在增大。在训练初始的时候，训练准确率可能会大幅度跳动，就像这幅图一样。这一方面跟学习率有关，有可能过大导致反向更新参数时参数来会波动。另一方面也跟数据集有关，数据集不够大，区分度不明显。但是训练到一定步数之后，准确率和损失值应该趋于稳定，否则此模型不适用于这组数据。

```
Epoch: 5 |train loss: 1.3321 验证集上准确率: 0.57621247
Epoch: 5 |train loss: 1.2282 验证集上准确率: 0.5877598
Epoch: 5 |train loss: 1.2480 验证集上准确率: 0.5993072
Epoch: 5 |train loss: 1.2361 验证集上准确率: 0.59584296
Epoch: 5 |train loss: 1.2745 验证集上准确率: 0.6120092
Epoch: 5 |train loss: 1.3817 验证集上准确率: 0.60277134
Epoch: 5 |train loss: 1.3266 验证集上准确率: 0.59237874
Epoch: 6 |train loss: 1.2204 验证集上准确率: 0.5935335
```

六、 扩展实验

(1) 修改代码，可视化查看经过 CNN 处理后的特征图。

(2) 调整超参数或者网络结构（比如增减层），在迭代一定的情况下，尽量提高精度，并对结果进行分析说明。

（扩展实验至少做一个）